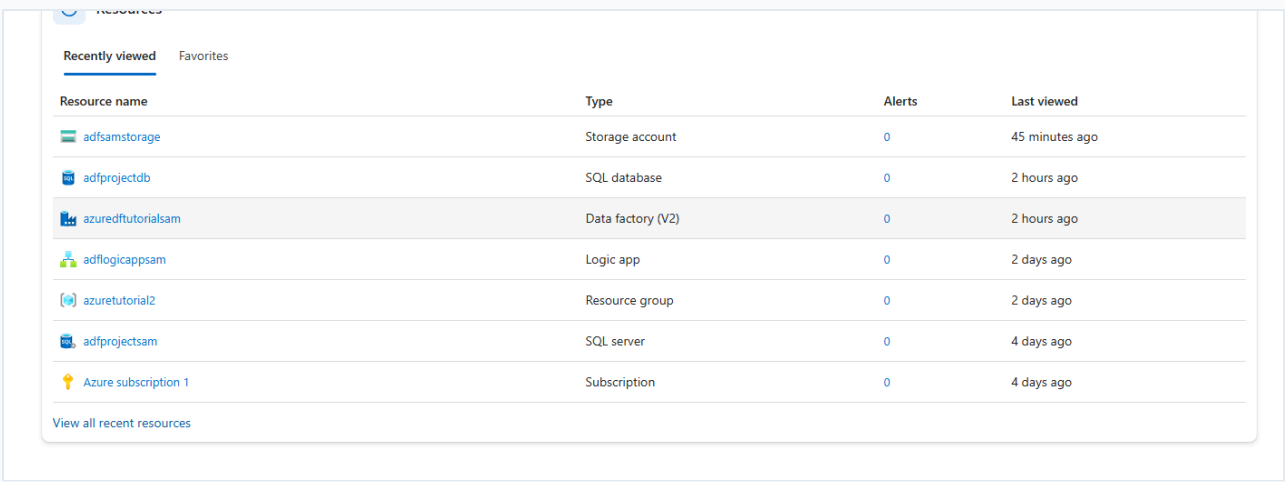
Azure Data Engineering Architecture Portfolio

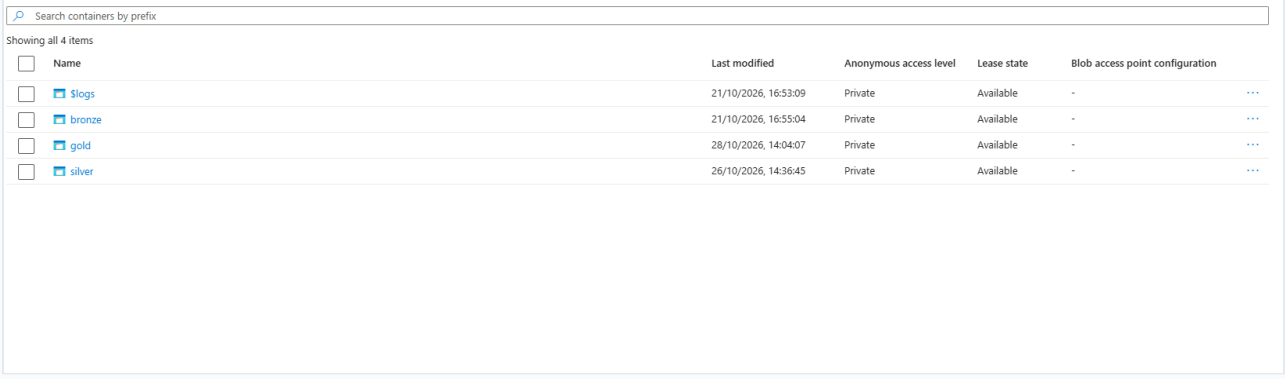
> **Project Overview:** This document serves as a technical artifact demonstrating a fully functional, end-to-end Data Lakehouse implementation using Microsoft Azure. The architecture ingests multi-source data (API, On-Premises, and Azure SQL), standardizes it through a Medallion architecture (Bronze, Silver, Gold), and orchestrates the entire workflow utilizing Azure Data Factory and Logic Apps.

1. Cloud Infrastructure & Storage Layout

The foundation of the project relies on Azure Blob Storage/Data Lake Gen2 configured with a strict Medallion architecture structure, alongside an Azure SQL Database serving as the transactional source system.



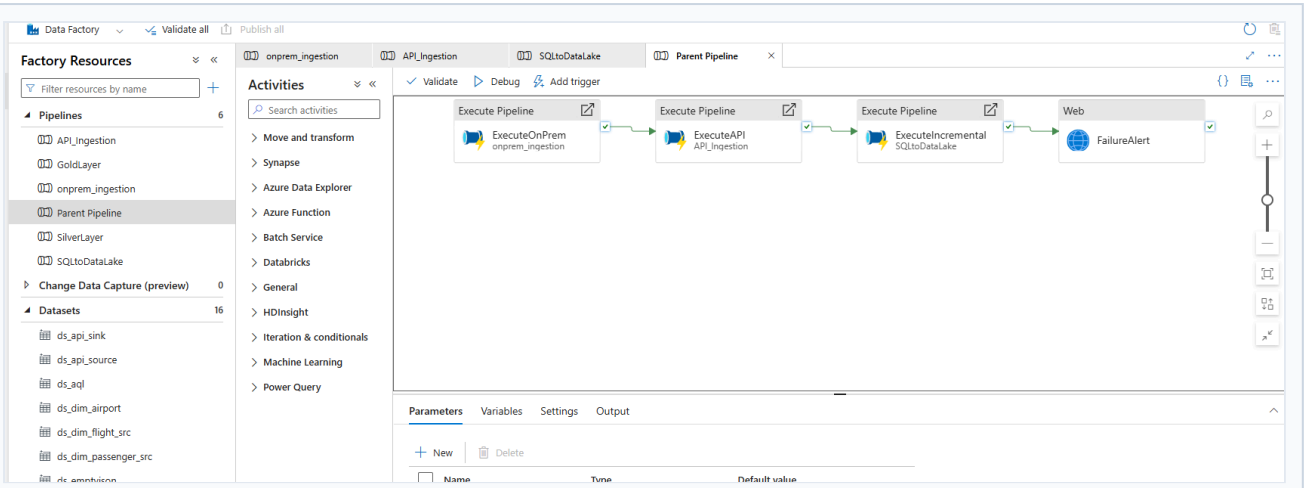
Deployed Azure Services (ADF, SQL Database, Storage Account, Logic Apps)



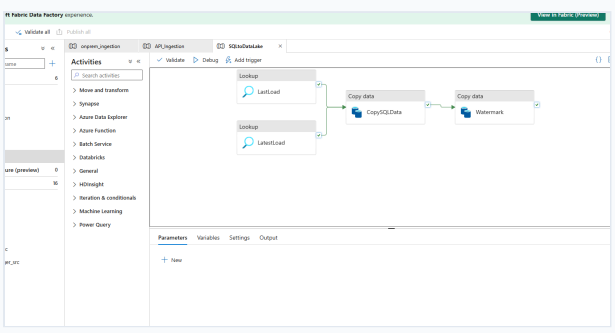
Data Lake Medallion Architecture (Bronze for raw ingestion, Silver for cleansed data, Gold for business-level aggregates)

2. Master Orchestration & Ingestion Logic

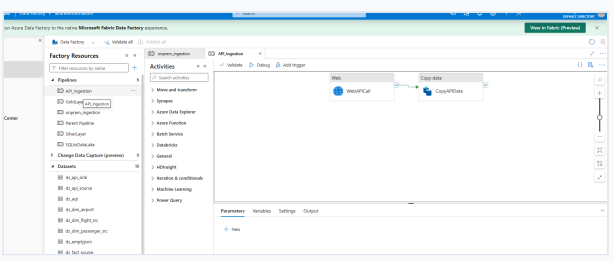
Pipeline execution is controlled by a centralized Parent Pipeline that triggers modular ingestion workflows. Data is securely pulled from Web APIs, parameterized on-premises file loops, and an incremental SQL ingestion framework utilizing dynamic High-Watermark tracking.



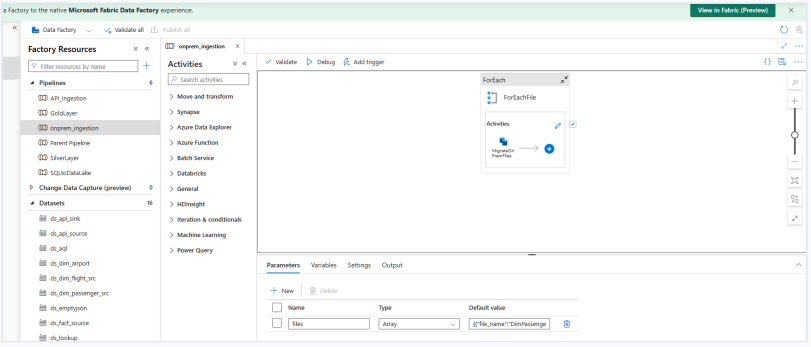
Parent Pipeline executing sequential ingestion and handling failure webhooks to Logic Apps



SQLToDataLake: Dynamic incremental load using parallel lookups



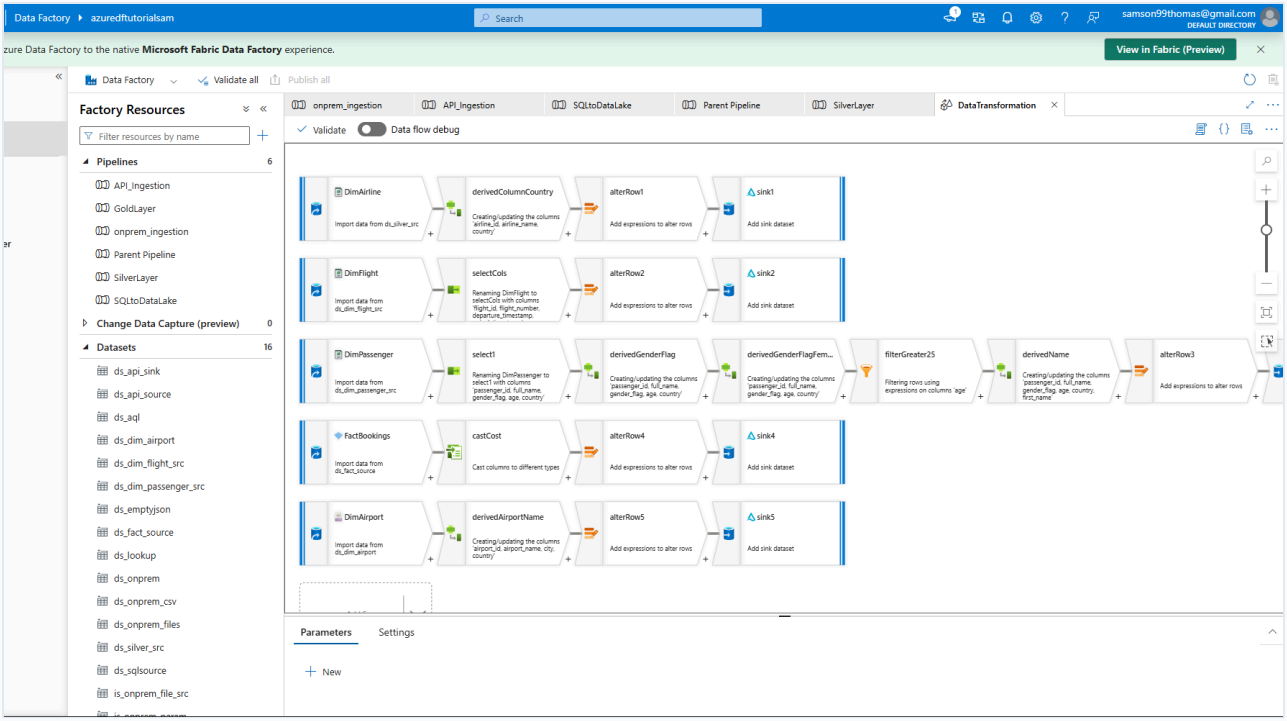
API_Ingestion: REST API extraction via Web and Copy Data activities



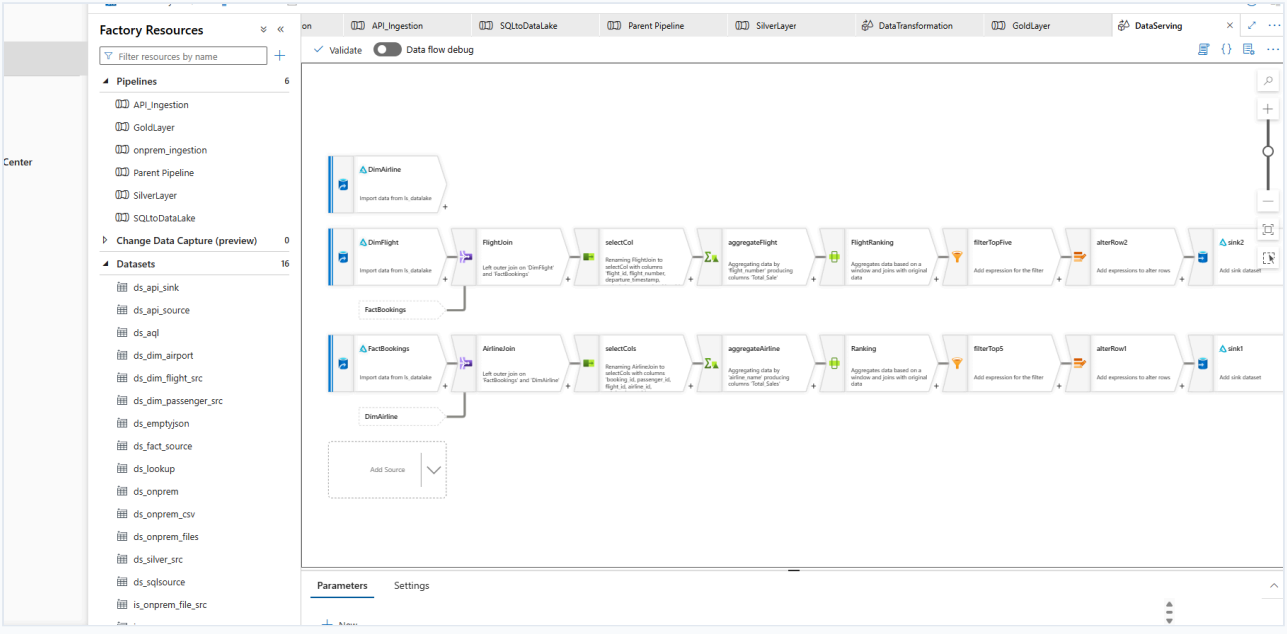
On-Premises Integration: Parameterized ForEach loop to migrate local files

3. Mapping Data Flows (Silver & Gold Layers)

Raw data is transformed natively in the cloud using ADF Mapping Data Flows. The Silver layer handles data typing, filtering (e.g., passenger demographics), and schema standardization. The Gold layer performs complex dimensional joins and windowed aggregations to surface top-performing metrics.



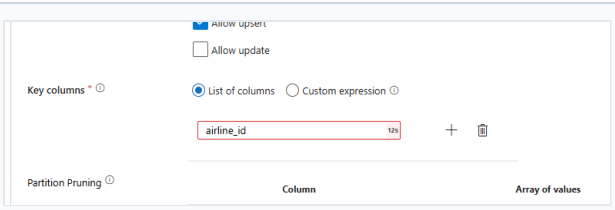
DataTransformation (Silver Layer): Multi-stream dimensional cleansing, casting, and derived logic



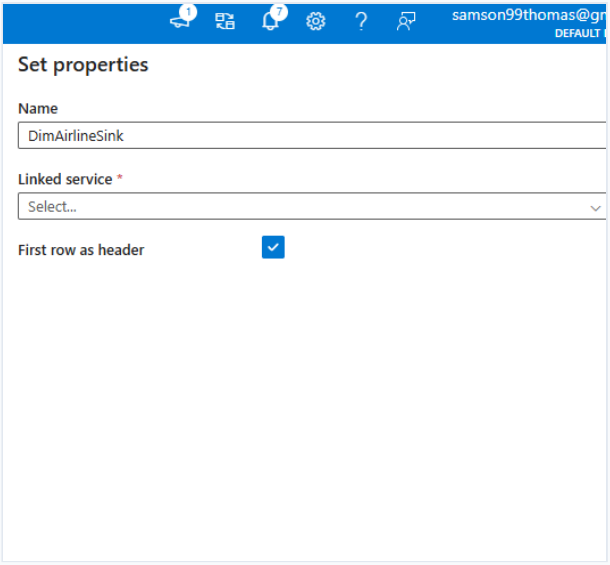
DataServing (Gold Layer): Fact-to-Dimension joins, aggregations, and Top 5 filtering

4. Advanced Configuration & Error Resolution

The pipeline architecture leverages robust Sink configurations to handle schema drift and database upserts. Cache sinks were strategically utilized during development to allow pipeline validation and modular testing without requiring physical endpoint instantiation.



Configuring explicit list-based Key Columns to support Alter Row Upsert policies



Standardizing linked service credentials and header mappings



Terminating development streams with structural sinks to satisfy ADF validation logic

